Project Report

Student 1: Lucy Lin
Student #: 1009876653

Student 2: Snow Shi
Student #: 1010143426

1. Purpose of Project

MyTix Java/JDBC command-line application with a fully designed and operational relational database. It models the core operations of an event-ticketing platform in the style of Ticketmaster. The system tracks venues and their physical seating layouts, events and their touring/repeating performances, organizer-managed pricing tiers, customer bookings, ticket cancellations, a peer-to-peer resale marketplace with an enforced price cap, and post-performance customer reviews.

Beyond simple data storage, the design's central technical challenge was guaranteeing a set of business rules at the database level rather than only in application code — most notably that a single seat can never be sold to two different customers for the same performance, that a customer can only cancel their own tickets, and that resale prices can never exceed an organizer-defined cap — so that these guarantees hold even under concurrent access or a bug in the application layer.

2. Conceptual Problems & Solutions

2.1. Ticket ownership history could not represent more than one resale

The first draft's TicketOwningHistory stored a single ownerId/previousOwnerId pair per ticket, with ticketId as the primary key. However, we later realized that because ticketId would functionally determine all other attributes, not 2 tuples can have the same ticket while having different ownerId and previousOwnerId, so they will be overwritten on each transfer. This directly conflicts with the requirement that "the system must retain the complete ownership history of every ticket" and with the sample-data requirement for at least one ticket that changes hands twice.

The fix was to create a dedicated historyId primary key for TicketOwnershipHistory(historyId, ticketId, sellerId, buyerId, transactionPrice, transactionDate), so each transfer is its own individual row rather than a mutable row shared by multiple transactions. Tickets.currentOwnerId separately tracks current ownership for fast lookups, while the history table answers "who has ever owned this ticket."

2.2. Guaranteeing "a seat is never double-booked" required deciding where the guarantee actually lives.

An application-level check ("is this seat free?") followed by an insert can introduce a race condition, in which two customers could both pass the check before either commits.

The solution was to place the actual guarantee at the schema level: UNIQUE(performanceId, seatId) on Tickets. This is also why Tickets deliberately stores performanceId directly rather than only deriving it via orderId → Orders.performanceId -- the constraint can only be declared if both columns live in the same table (documented explicitly as a justified BCNF exception in the normalization section). The application layer still performs an availability check first, purely to give the user a clean error message instead of a raw constraint-violation exception; the constraint is what actually makes double-booking structurally impossible.

2.3 General-admission capacity has the same race condition, but no UNIQUE constraint can express it.

Since GA tickets all have seatId = NULL, there's no per-seat row to constrain.

Instead, our solution is that bookGeneralAdmission locks the relevant Sections row with SELECT ... FOR UPDATE before counting sold tickets and comparing against standingCapacity, serializing any two transactions trying to book the same GA section concurrently. This is weaker than a direct UNIQUE constraint in the schema, but it can still achieve the concurrency we want.

2.4 Some business rules span two rows in different tables and cannot be expressed as a CHECK at all.

Two examples:
(a) a section assigned to a price tier for a performance must belong to the same venue as that performance (trg_section_venue_match)
(b) a ticket must have a seatId if and only if its section is reserved seating, never if it's general admission (trg_ticket_seat_consistency).

Both were implemented as BEFORE INSERT triggers rather than CHECK constraints, since MySQL's CHECK is limited to a single row's own columns. The trigger solution allows us to check that the values being inserted indeed meet the requirements.

2.5. Not every constraint can live in the database.

Whether a customer actually attended a performance (held a non-cancelled ticket for a performance that has already happened) before allowing a review is a dynamic, time-dependent check (p.dateTime < NOW()) that can't be expressed as a static CHECK, and MySQL triggers cannot reference NOW()-dependent logic the same way application code can cleanly act on it with a readable error message.

This validation was deliberately placed in ReviewOperations (Java) rather than a trigger, while the static rule it depends on — one review per customer per performance — is still enforced at the schema level via UNIQUE(customerId, performanceId) on Comments. This reflects a general principle that we applied throughout the design: static, row-local rules go in CHECK/UNIQUE; cross-table rules go in triggers; time-dependent or purely informational rules go in the application layer, with the schema still enforcing whatever can be expressed there as a backstop.

3. Assumptions

3.1 General Assumptions

- Only 1 organizer per event
- A customer can list the same ticket for resale multiple times (not at the same time) [i.e. customer lists the ticket → customer withdraws → customer re-lists the ticket → …]
- An event can exist in multiple venues (as in it can take up several venues)
- Every event should have a title (mandatory)
- A resale price cap for an event stays the same no matter how many resale transactions go on
- Events and performances have their own required metadata, such as the event/performance name, description, etc.
- Email is the effective login identity. The schema has no password/authentication table, so this is a demo-style login, not production security.
- Performance lookups use either an ID or an exact performance name plus date/time to the minute (YYYY-MM-DDTHH:MM).
- Availability is computed from Active tickets only — a cancelled ticket's seat/capacity is automatically freed simply because it no longer counts toward "sold." This is the foundational rule that Q1, Q4, Q5, Q6, Q7, and R7 all rely on, so it's stated once here rather than repeated per query/report.
- Refunds are recorded as audit entries in the database (e.g. ticket status change, cancellation timestamp); no real payment processor is contacted.

3.2 Query Assumptions

Q1

- Only showing upcoming performances that still have seats/capacity available

Q2

- Postal “adjacency” is data-driven through PostalCodeAdjacency; it must be populated with valid neighbouring postal prefixes.

Q3

- Venue address is treated as an exact match.

Q6

- It’s reporting for a single query; a performance ID should be given as input

3.3 Report Assumptions

R1

- What counts as a “sold” ticket? A cancelled ticket got a full refund, so it shouldn't count toward gross revenue - only Tickets.status = 'Active' should be counted.
- What "date range" means. We interpret it as the performance's date (revenue attributed to when the show happens). So it only considers tickets for performances that happen in that date range. (From another perspective, this can ensure consistency. A customer can buy and cancel a ticket, with a full refund. However, if we only consider performances that will happen in this date range, then the refund window is minimized, and the reported number is less likely to be inaccurate due to a large amount of refunds

R2

- This report will count all performances and events, even the cancelled ones

R3

- Will interpret the question requirement as requiring a report ranking organizers by gross revenue overall, by gross revenue per country, and finally ranking at a specific city (and not “per city”)
- Gross revenue adds up the prices of all sold tickets (with active status to indicate it has not been cancelled and refunded) up to this date
- overall → show everyone, including $0 income organizers
- per-country / by-city → only show who actually participated (those with 0 income in that country/city, as well as countries/city with no organizer making any income, will not show up in the report)

R6

- The assumption is that when we find customers with the most cancelled tickets in a year, we are referring to tickets being cancelled in that year (unrelated to performance time)
- We are not ranking them, but only giving the customer/organizer with the most cancellations (there could be multiple of them with the same cancel amount)
- If a year has no ticket or performance cancellations, the report result table will be empty

R9

- Using Java-side text processing approach

3.4 Operation Assumptions

Delete User

- When deleting users, we assume that when certain history/record, like ticket purchase history, order history, etc., still exists, then we cannot delete the user (in order to preserve all history as required by the requirements)

Booking

- A customer's most recently added payment method is automatically used for bookings and resale purchases.

Cancellation

- Customer cancellation is permitted only for active tickets bought by that customer and at least seven days before the performance.
- Cancelled tickets free their seats/capacity because availability counts only active tickets.

Event / Pricing Management

- Organizers can only manage events and performances they created.
- Tier prices cannot change once any ticket has been issued for that tier, even if it was later cancelled.
- Section name and tier name are treated as exact matches when an organizer references them (e.g. assigning sections to tiers).

Reviews

- Event title is treated as an exact match when identifying which event a review belongs to.
- Reviews are allowed only when the customer currently holds an active ticket for a past performance of that event; one review per customer/performance is enforced.

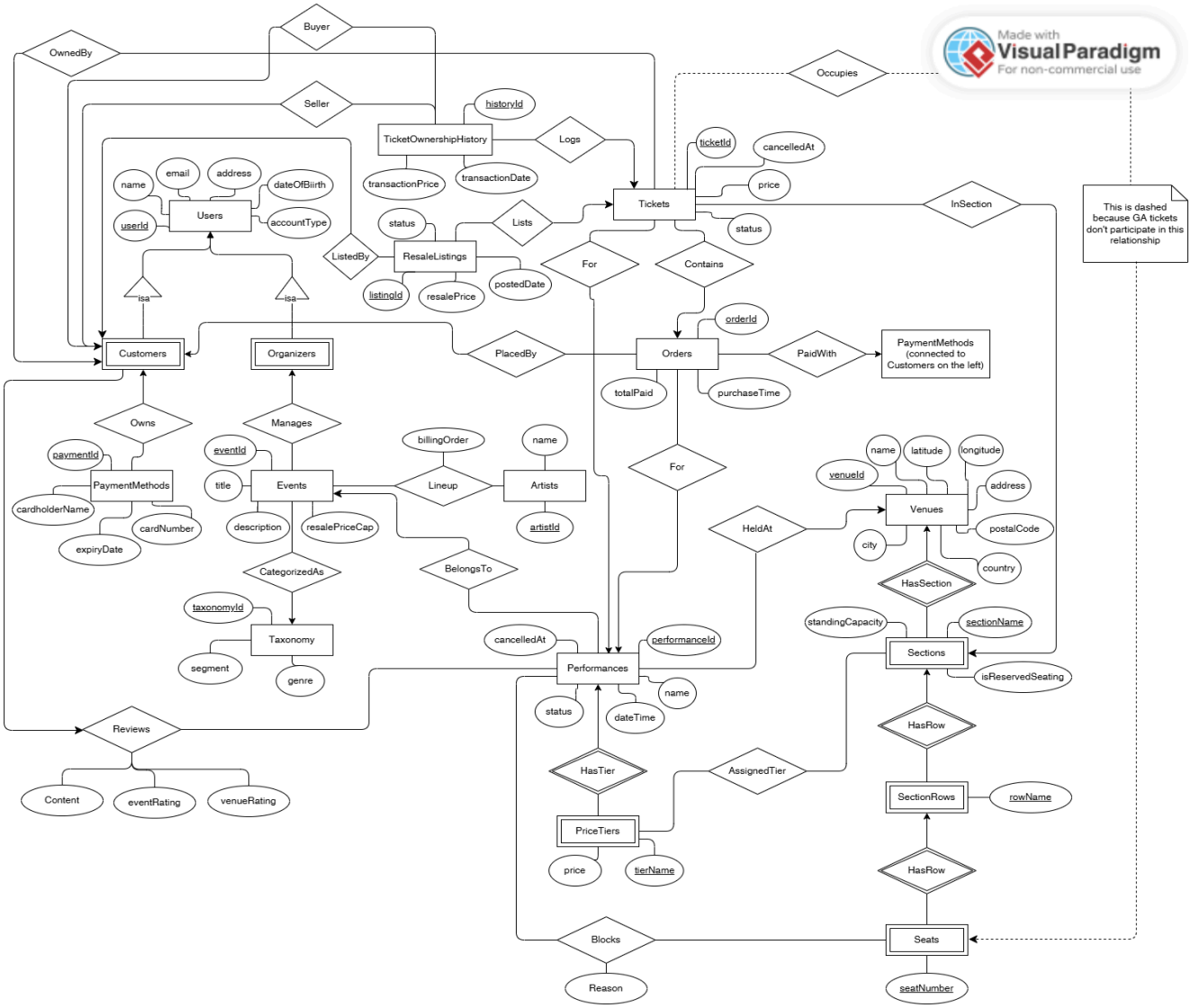
Resale

- Resale purchases use the customer's saved (most recently added) payment method and obey the event's resale price cap.

3.5 Organizer Toolkit Assumptions

- The pricing toolkit uses historical comparable data and simple averages/heuristics, so its recommendations are estimates rather than guarantees.

4. ER Diagram:



Also, link to the diagram:
https://online.visual-paradigm.com/share.jsp?id=343637393930312d31

5. Relation Schema, Keys and Non-Trivial FDs:

Relation	Key(s) (everything inside 1 {...} is a single key)	Nontrivial FDs	Normal Form
Users	(userId} {email}	userId → name, email, address, dateOfBirth, accountType	BCNF

		email → userId, name, address, dateOfBirth, accountType	
Customers	{customerId}	none (single attribute relation)	BCNF
Organizers	{organizerId}	none (single attribute relation)	BCNF
PaymentMethods	{paymentId}	paymentId → customerId, cardholderName, cardNumber, expiryDate	BCNF
Taxonomy	{taxonomyId} {segment, genre}	taxonomyId → segment, genre segment, genre → taxonomyId	BCNF
Events	{eventId}	eventId → organizerId, taxonomyId, title, description, resalePriceCap	BCNF
Artists	{artistId}	artistId → name	BCNF
EventLineups	{artistId, eventId}	artistId, eventId → billingOrder	BCNF
Venues	{venueId}	venueId → Name, latitude, longitude, address, postalCode, city, country	BCNF (see assumption below)
Sections	{sectionId} {venueId, sectionName}	sectionId → venueId, sectionName, isReservedSeating, standingCapacity venueId, sectionName → sectionId, isReservedSeating, standingCapacity	BCNF
SectionRows	{rowId} {sectionId, rowName}	rowId → sectionId, rowName sectionId, rowName → rowId	BCNF
Seats	{seatId} {rowId, seatNumber}	seatId → rowId, seatNumber rowId, seatNumber → seatId	BCNF
Performances	{performanceId}	performanceId → eventId, venueId, name, dateTime, status, cancelledAt	BCNF
PriceTiers	{tierId} {performanceId, tierName}	tierId → performanceId, tierName, price performanceId, tierName → tierId, price	BCNF

PerformanceSectionAssignments	{sectionId, performanceId}	sectionId, performanceId → tierId	BCNF
BlockedSeats	{performanceId, seatId}	performanceId, seatId → reason	BCNF
Orders	{orderId}	orderId → customerId, performanceId, paymentId, purchaseTime, totalPaid	BCNF
Tickets	{ticketId}	ticketId → orderId, performanceId, sectionId, seatId, price, currentOwnerId, status, cancelledAt orderId → performanceId	3NF
TicketOwnershipHistory	{historyId}	historyId → ticketId, sellerId, buyerId, transactionPrice, transactionDate	BCNF
ResaleListings	{listingId}	listingId → ticketId, sellerId, resalePrice, postedDate, status	BCNF
Comments	{commentId} {customerId, performanceId}	commentId → customerId, performanceId, content, eventRating, venueRating customerId, performanceId → commentId, content, eventRating, venueRating	BCNF

6. 3NF / BCNF Proofs

Note that the following proofs are justifications for our final schema. It may use relations from draft 1 to demonstrate our decompositions.

6.1 Proof 1: Orders

Draft 1 relation:
Orders(orderId, customerId, performanceId, eventId, purchaseTime, paymentId, totalPaid)

FDs:
orderId → customerId, performanceId, eventId, purchaseTime, paymentId, totalPaid
performanceId → eventId

Step 1:
performanceId$^+$ = {performanceId, eventId}.

It does not functionally determine all attributes of Orders, so performanceId is not a superkey → BCNF violated.

Step 2:
Decompose into:

- R1 = X^+ = {performanceId, eventId}
- R2 = $(R - X^+) \cup X$ = {orderId, customerId, performanceId, purchaseTime, paymentId, totalPaid}.

Step 3:
Project the FDs:

- R1: performanceId → eventId (key={performanceId} → R1 is BCNF)
- R2: orderId → customerId, performanceId, purchaseTime, paymentId, totalPaid (key={orderId} → R2 is BCNF)

Result:

- R1 is already a part of Performances.
- R2 is exactly the final Orders relation (with paymentMethod being the paymentId). Thus, the current Orders is BCNF.

6.2 Proof 2: Performances / PriceTiers

Draft 1 relation:
PerformanceInfo(performanceId, eventId, venueId, name, dateTime, status, cancelledAt, sectionName, price)

FDs:
performanceId → eventId, venueId, name, dateTime, status, cancelledAt
performanceId, sectionName → price

Step 1:
$performanceId^+$ = {performanceId, eventId, venueId, name, dateTime, status, cancelledAt}

It does not functionally determine all attributes of the relation (missing sectionName, price), so performanceId is not a superkey → BCNF violated.

Step 2:
Decompose into:

- R1 = X^+ = {performanceId, eventId, venueId, name, dateTime, status, cancelledAt}
- R2 = $(R - X^+) \cup X$ = {performanceId, sectionName, price}.

Step 3:
Project the FDs:

- R1: performanceId → eventId, venueId, dateTime (key={performanceId} → is BCNF)
- R2: performanceId, sectionName → price (key={performanceId, sectionName} → is BCNF)

Result:
- R1 becomes Performances.
- R2 becomes PriceTiers (with one further real-world refinement beyond the BCNF decomposition:)
    - Since the project handout requires a tier's price to be independent of any one section's name (multiple sections can share a tier, e.g. "P1"), sectionName in R2 was replaced by a tierName identifying the tier itself, and a separate relation called PerformanceSectionAssignments records which sections are assigned to which tier per performance
    - Proof 2.1:
        - PerformanceSectionAssignments(sectionId, performanceId, tierId)
        - FD: sectionId, performanceId → tierId
        - Its key is {sectionId, performanceId}, so it is BCNF (as all FD's left side is a key)

6.3 Proof 3: PerformanceSectionAssignments / BlockedSeats

Draft 1:
PerformanceSectionInfo(sectionId, performanceId, priceTier, seatId, reason)
(where seatId and reason for blocked seats)

FDs:
sectionId, performanceId → priceTier
performanceId, seatId → reason, sectionId

Step 1:
$(\text{sectionId, performanceId})^+$ = {sectionId, performanceId, priceTier}
It does not functionally determine all attributes (missing seatId, reason), so {sectionId, performanceId} is not a superkey → BCNF violated

Step 2:
Decompose into:
- R1 = X^+ = {sectionId, performanceId, priceTier}
    - (This is the same relation as R1 in Proof 2 — PerformanceSectionAssignments, with priceTier renamed to tierId)
- R2 = $(R - X^+) \cup X$ = {sectionId, performanceId, seatId, reason}.

Step 3:
Project the FDs:

- R1: sectionId, performanceId → priceTier (key={sectionId, performanceId} → BCNF)
- R2: performanceId, seatId → reason, sectionId
    - {performanceId, seatId} is already a superkey of R2, making sectionId a redundant attribute within R2, because once you know the blocked seat's seatId and performanceId, you will be able to find the specific sectionId in the Sections, SectionRows, and Seats relations.
    - Dropping it gives the minimal R2' = {performanceId, seatId, reason} (key= {performanceId, seatId} → BCNF)

Result:
- R1 merges into PerformanceSectionAssignments (see Proof 2 > Proof 2.1 for more info), which is BCNF
- R2' is exactly the final BlockedSeats(performanceId, seatId, reason).

6.4 BCNF/3NF assumptions - Tickets / Venues

1) Tickets(ticketId, orderId, performanceId, sectionId, seatId, price, currentOwnerId, status, cancelledAt):

This relation has the following FDs:
ticketId → orderId, performanceId, sectionId, seatId, price, currentOwnerId, status, cancelledAt
orderId → performanceId

Note that 'orderId → performanceId' is an FD where orderId is not a superkey and performanceId is not prime (the only candidate key is ticketId), violating both BCNF and 3NF. This is kept deliberately because UNIQUE(performanceId, seatId) on a single table is the only DB-enforced mechanism available to make double-booking structurally impossible, which the assignment explicitly requires. So, we are including performanceId here.

2) Venues(venueId, name, latitude, longitude, address, postalCode, city, country):

This relation is said above to have those FDs:
venueId → Name, latitude, longitude, address, postalCode, city, country

But in real life, several FDs will hold w.r.t the locations:
postalCode → city, country
Address → postalCode, city, country, latitude, longitude
…

So for this assignment, we are treating postalCode, city, and country as independently entered attributes rather than modelling a governed postal-code-to-region mapping. So, under this assumption, those real-life FDs will not hold, and Venues remains in BCNF.

6.5 All other proofs

There are still several more relations above: Users, Customers, Organizers, PaymentMethods, Taxonomy, Events, Artists, EventLineups, Venues, Sections, SectionRows, Seats, TicketOwnershipHistory, ResaleListings, Comments.

For each of those relations, every nontrivial FD's determinant (i.e. the X in $X \rightarrow A$) is a candidate key for that relation, so by definition every relation satisfies BCNF trivially.

7. DDL Statement

The complete DDL is in sql/schema.sql (as it is quite long, we will omit it from the report).

7.1 Primary keys

Every relation uses an INT AUTO_INCREMENT primary key (userId, venueId, ticketId, etc.) rather than a natural key, even where a natural composite key exists (e.g. (sectionId, sectionName) for Sections). This keeps foreign keys elsewhere in the schema single-column and stable even if a natural attribute (like a venue's name) is later corrected – the natural key is instead preserved as a UNIQUE constraint alongside the dedicated key, so both uniqueness and referential simplicity are maintained. The exceptions are pure many-to-many relationship tables (EventLineups, PerformanceSectionAssignments, BlockedSeats), which use their natural composite key directly as the primary key since there's no independent identity to the association itself.

7.2 Money and coordinates

All monetary values (PriceTiers.price, Tickets.price, Orders.totalPaid, etc.) use DECIMAL(10,2) rather than FLOAT/DOUBLE, since floating-point binary representation cannot exactly represent most base-10 currency values, which is unacceptable for anything involving payments. Venues.latitude/longitude use DECIMAL(9,6), giving roughly 0.1 m precision at the equator – more than sufficient for a venue location and avoiding floating-point drift in the ST_Distance_Sphere distance calculations used by the location search.

7.3 Enumerated values

Fixed, small vocabularies (Users.accountType, Performances.status, Tickets.status, ResaleListings.status, EventLineups.billingOrder) use MySQL's ENUM type rather than a free-text VARCHAR or a separate lookup table. This was a deliberate choice: these sets are small, fixed by the business domain (not user-extensible), and an ENUM gets validation "for free" without an extra join. Taxonomy (segment/genre), by contrast, is a separate lookup table

rather than an ENUM, since the assignment requires it to mirror Ticketmaster's actual (larger, evolving) taxonomy – a set that isn't reasonably hardcoded into the type system.

7.4 CHECK constraints

Used wherever a rule is a static function of a single row's own columns: rating bounds (chk_comments_event_rating/venue_rating: BETWEEN 1 AND 5), non-negative prices (chk_ticket_price_non_neg and siblings), the resale cap floor (chk_resale_price_cap_non_neg: resalePriceCap >= 1.00), and the two "nullability depends on a sibling column" rules – chk_standing_capacity (a section has a standingCapacity if and only if it's general admission) and chk_tickets_cancelled_at/chk_performances_cancelled_at (a cancelledAt timestamp exists if and only if the row's status is a cancelled state).

7.5 UNIQUE constraints

Every composite natural key is enforced using the UNIQUE constraints: (venueId, sectionName), (sectionId, rowName), (rowId, seatNumber), (performanceId, tierName), (customerId, performanceId), and (performanceId, seatId).

7.6 Foreign keys and ON DELETE behaviour

Deliberately not CASCADE everywhere. Tables representing historical fact -- Orders, Tickets, TicketOwnershipHistory, ResaleListings, Events (via its FK from Organizers) -- default to RESTRICT, meaning a customer or organizer with any real transaction history cannot be deleted at all. It directly implements the requirement to "keep a complete history of the orders and tickets they have purchased" and "maintain the list of events they manage" — the database itself refuses to destroy that history via UserOperations.deleteUser, which only succeeds for accounts with no retained history.

CASCADE is used only where child rows have no independent meaning once the parent is gone (PaymentMethods when a Customer is deleted, Seats/SectionRows/Sections cascading from Venues, TicketOwnershipHistory/ResaleListings cascading from Tickets).

7.7 Views

SectionAvailability, PerformanceCheapestPrice, AvailableSeatsByPerformance, and EventPerformanceLocations are not required by the ER model itself but were added to centralize seat-availability and location-join logic that multiple queries and reports depend on (Q1, Q6, Q7, R1, R2), so that a change to "what counts as available" only needs to happen in one place. Note that those are not part of our designed database schema; they are only helpers to help with our implementations and queries.